

Intermediate Programmer - Final Project Brief

Inventory & Order Management System

PT Neuronworks Indonesia · Participant Guide · Edisi 1.0 Program Pengembangan Kompetensi Programmer · Oktober 2026

Model pengerjaan	Individual - source code, repository, dan evidence dibuat sendiri.
Produk	Aplikasi web Inventory & Order Management System, 3 peran, multi-gudang.
Teknologi inti	HTML, CSS, Vanilla JS + Fetch API, PHP 8.2+ Native OOP berlapis, MySQL 8, Docker Compose, PHPUnit, static analysis.
Alur inti	Login -> Master Data -> Purchase Order -> Sales Order -> Stock Ledger -> Dashboard/Laporan -> Logout.
Nilai project	Minimal 80 dan tidak terkena critical failure.

0. Baca Ini Sebelum Memulai

Lima ketentuan teknis yang tidak dapat diganti

- Backend menggunakan PHP Native tanpa framework, menerapkan OOP dengan pemisahan Controller / Service / Repository, dan Dependency Inversion pada boundary repository.
- Interaksi frontend menggunakan Vanilla JavaScript tanpa framework JS, ditambah minimal satu endpoint JSON API.
- Aplikasi dan database wajib dapat dijalankan menggunakan Docker Compose.
- Project wajib memiliki unit test *dan* integration test yang memenuhi standar minimum pada brief ini.
- Project wajib menyertakan class diagram, Architecture Decision Record, dan refactoring log sebagai bukti.

Cara membaca label

Label	Arti
WAJIB	Harus dipenuhi dan berfungsi agar project dapat dinilai.
DIPERBOLEHKAN	Boleh digunakan dalam batas yang dinyatakan.
BONUS	Dinilai setelah seluruh requirement wajib stabil.
BUKTI	Artefak atau demo yang memverifikasi pekerjaan peserta.
REFLEKSI	Dokumen penalaran desain - ADR, refactor log, audit SRP, tech-debt register.
PERINGATAN	Batas yang sengaja dijaga agar peserta tidak melakukan over-engineering.

Peringatan Level Intermediate menambah kedalaman desain, bukan kerumitan yang tidak perlu. Solusi yang lebih rumit tanpa alasan jelas (pattern yang dipasang tanpa dipahami, layer tambahan yang tidak menyelesaikan masalah nyata) dinilai negatif pada area code quality, sama seperti kode yang berantakan.

1. Project yang Harus Dibangun

Tim gudang dan sales membutuhkan aplikasi untuk mencatat produk, mengelola stok di beberapa gudang, memproses pembelian dari supplier serta penjualan ke customer, dan memastikan angka stok selalu bisa dipertanggungjawabkan - termasuk saat dua proses berjalan bersamaan. Aplikasi digunakan oleh tiga peran dengan tanggung jawab yang sengaja dipisah agar tidak ada satu peran yang bisa membuat sekaligus menyetujui transaksi yang sama.

Tujuan Membuktikan bahwa peserta mampu merancang arsitektur berlapis yang dapat diuji, menjaga integritas data pada operasi konkuren, menerapkan prinsip Clean Code dan Clean Architecture secara nyata - bukan sekadar istilah di README - serta mengomunikasikan keputusan desain secara jelas kepada pihak lain.

1.1 Alur utama

1. Admin login, mengelola user (Sales / Warehouse Staff) dan master data (produk, kategori, gudang, supplier, customer).
2. Sales login, membuat Sales Order berstatus Draft berdasarkan katalog dan stok yang tersedia, lalu mengajukannya sehingga statusnya berubah menjadi PendingApproval.
3. Admin meninjau dan menyetujui atau menolak Sales Order. Sales tidak dapat menyetujui order miliknya sendiri.
4. Warehouse Staff memproses goods issue untuk Sales Order yang Approved, mengurangi stok melalui transaksi yang aman dari race condition; order menjadi Fulfilled.
5. Saat stok rendah, Admin atau Warehouse Staff membuat Purchase Order ke supplier; ketika barang datang, Warehouse Staff mencatat goods receipt yang menambah stok.
6. Setiap pergerakan stok tercatat pada stock ledger; dashboard dan laporan dihitung dari data ini, bukan angka statis.
7. User logout dan tidak dapat membuka halaman terlindungi tanpa login kembali.

1.2 Peran dan segregation of duties

Aktivitas	Admin	Sales	Warehouse Staff
Login, logout, profil sendiri	Boleh	Boleh	Boleh
Mengelola user	Boleh	Tidak	Tidak
Mengelola master data (produk/ gudang/supplier/customer)	Boleh	Hanya melihat katalog	Hanya melihat produk & stok
Membuat & mengajukan Sales Order	Boleh	Boleh, milik sendiri	Tidak
Menyetujui / menolak Sales Order	Boleh	Tidak, meski order miliknya sendiri	Tidak
Membuat Purchase Order	Boleh	Tidak	Boleh mengusulkan
Memproses goods receipt (PO)	Boleh	Tidak	Boleh
Memproses goods issue (SO)	Boleh	Tidak	Boleh
Melihat dashboard	Seluruh data	Ringkasan order miliknya	Ringkasan stok & fulfillment
Mengunduh laporan (CSV)	Boleh	Order miliknya	Laporan stok

Keputusan proses Segregation of duties: Sales yang membuat order tidak boleh menyetujui order yang sama, meskipun endpoint approve tetap tersedia di aplikasi untuk digunakan peran Admin. Aturan ini wajib ditegakkan pada authorization layer di server, bukan hanya disembunyikan lewat UI - mencerminkan praktik nyata pada sistem inventori dan keuangan agar satu orang tidak dapat menyalahgunakan alur transaksi sendirian.

1.3 Data dan nilai yang digunakan

Entity	Field minimum	Nilai tetap
User	Nama, email, password, role, status aktif, timestamps	Admin / Sales / WarehouseStaff
Warehouse	Nama, lokasi, status aktif	-
Category	Nama, deskripsi	-
Product	SKU (unik), nama, kategori, unit, harga beli, harga jual, reorder point, gambar (opsional), status aktif	-
ProductStock	Produk, gudang, quantity, updated_at	quantity >= 0
Supplier / Customer	Nama, kontak, alamat, status aktif	-
PurchaseOrder + Item	Supplier, gudang tujuan, status, tanggal order, item (produk, qty, harga beli)	Draft/Ordered/PartiallyReceived/Received/Cancelled
SalesOrder + Item	Customer, dibuat oleh, disetujui oleh, gudang asal, status, item (produk, qty, harga jual)	Draft/PendingApproval/Approved/Fulfilled/Cancelled
StockLedger	Produk, gudang, tipe pergerakan, quantity, referensi (PO/SO id), dilakukan oleh, timestamp	Receipt/Issue/Adjustment

Alur status Sales Order: Draft -> PendingApproval -> Approved -> Fulfilled (atau Cancelled pada tahap manapun sebelum Fulfilled).

Keputusan data Produk, supplier, dan customer dinonaktifkan, bukan dihapus permanen. Stok **tidak pernah** diubah langsung oleh UI - hanya lewat service

yang menulis baris StockLedger lalu memperbarui ProductStock dalam satu transaksi. Ini menjaga riwayat dan membuat setiap perubahan angka stok bisa ditelusuri ke satu baris ledger.

2. Requirement Wajib

Urutan pembangunan Selesaikan vertical slice dahulu: login -> master data -> purchase order -> sales order -> dashboard/laporan. Tambahkan API, upload gambar, dan scheduled job setelah alur transaksi inti stabil.

2.1 Autentikasi & User

AUTH-01 Login dan Session

User masuk menggunakan email dan password yang valid; sesi menentukan tampilan sesuai role.

Ketentuan minimum: - Login valid mengarah ke dashboard sesuai role (Admin/Sales/Warehouse Staff). - Kredensial salah menampilkan pesan aman tanpa menjelaskan bagian yang salah; user tidak aktif tidak dapat login. - Halaman terlindungi tidak dapat diakses tanpa session; ID session diperbarui setelah login. - Password disimpan dengan `password_hash()` dan diverifikasi dengan `password_verify()`.

Bukti: Demo login tiga role, login gagal, dan akses halaman terlindungi tanpa session.

AUTH-02 Logout

User dapat mengakhiri session dari aplikasi.

Ketentuan minimum: - Logout menghapus data autentikasi pada session; URL terlindungi tidak dapat dibuka kembali tanpa login.

Bukti: Demo logout dan percobaan membuka kembali halaman terlindungi.

USR-01 Manajemen User (3 role)

Admin mengelola akun Sales dan Warehouse Staff.

Ketentuan minimum: - Admin dapat menambah, melihat, mengubah, serta mengaktifkan/nonaktifkan user; email unik. - Role hanya Admin, Sales, atau Warehouse

Staff; tidak ada public registration. - Sales dan Warehouse Staff tidak dapat membuka halaman atau endpoint administrasi user.

Bukti: CRUD terbatas user, validasi email duplikat, uji akses sebagai Sales dan sebagai Warehouse Staff.

2.2 Master Data

PRD-01 Produk, Kategori & Reorder Point

Admin mengelola katalog produk yang dipakai di seluruh alur transaksi.

Ketentuan minimum: - SKU unik; kategori, harga beli, harga jual, unit, dan reorder point tervalidasi (angka ≥ 0). - Produk yang sudah dipakai pada order hanya dapat dinonaktifkan, bukan dihapus. - Gambar produk (opsional) diunggah dengan validasi tipe dan ukuran file, disimpan dengan nama acak agar tidak dapat ditebak.

Bukti: Demo create/edit/nonaktif, validasi reorder point, dan upload gambar dengan file tidak valid.

WH-01 Gudang & Stok Multi-Lokasi

Stok satu produk dapat berbeda di setiap gudang.

Ketentuan minimum: - Admin mengelola daftar gudang; setiap produk memiliki baris stok per gudang. - Tampilan stok menunjukkan total dan rincian per gudang.

Bukti: Demo satu produk dengan stok berbeda di dua gudang.

2.3 Purchase Order & Penerimaan Barang

P0-01 Purchase Order & Goods Receipt

Admin atau Warehouse Staff membuat Purchase Order ke supplier ketika stok rendah, lalu mencatat penerimaan barang.

Ketentuan minimum: - PO memiliki supplier, gudang tujuan, item (produk, qty, harga beli); status mengikuti alur **Draft** -> **Ordered** -> **PartiallyReceived/Received** -> **Cancelled**. - Goods receipt menambah ProductStock **dan** menulis baris StockLedger bertipe Receipt dalam satu transaksi (lihat **ARCH-02**). - Penerimaan sebagian (partial) diperbolehkan; sisa qty yang belum diterima tetap tercatat.

Bukti: Demo create PO, goods receipt penuh dan sebagian, serta isi StockLedger yang dihasilkan.

2.4 Sales Order & Pengeluaran Stok

S0-01 Sales Order, Approval & Goods Issue

Sales membuat Sales Order, Admin menyetujui, Warehouse Staff memenuhi.

Ketentuan minimum: - SO mengikuti alur `Draft -> PendingApproval -> Approved -> Fulfilled`, atau `Cancelled` pada tahap manapun sebelum Fulfilled. - Authorization approve diperiksa di server: Sales tidak dapat menyetujui order - termasuk order miliknya sendiri. - Goods issue hanya dapat diproses untuk SO berstatus Approved, dan ditolak jika stok tersedia tidak mencukupi. - Goods issue mengurangi ProductStock **dan** menulis StockLedger bertipe Issue dalam satu transaksi yang aman dari race condition (lihat `ARCH-02`).

Bukti: Demo alur lengkap Draft->Fulfilled, uji Sales mencoba approve, dan uji goods issue saat stok tidak cukup.

2.5 Daftar, Pencarian, Dashboard & Laporan

VIEW-01 Daftar, Detail & Empty State

Ketentuan minimum: - Produk, PO, dan SO tampil sebagai daftar dan halaman detail sesuai role; keadaan tanpa data menampilkan empty state informatif.

Bukti: Screenshot berisi data dan tanpa data.

FIND-01 Search, Filter, Sort & Pagination

Ketentuan minimum: - Produk: pencarian nama/SKU, filter kategori dan status stok (low stock/normal). - Order (PO & SO): pencarian nomor/pihak terkait, filter status, sort tanggal naik/turun. - Daftar utama menggunakan pagination 10 data per halaman; filter tetap aktif saat berpindah halaman. - Seed menyediakan minimal 30 produk dan 25 order gabungan agar pagination dapat diuji.

Bukti: Demo kombinasi search/filter/sort dan perpindahan minimal dua halaman.

DASH-01 Dashboard sesuai Hak Akses

Ketentuan minimum: - Admin melihat nilai inventori, produk di bawah reorder point, dan order pending per status. - Sales melihat ringkasan order miliknya per status; Warehouse Staff melihat antrean goods receipt/issue dan produk low-stock. - Seluruh angka berasal dari query agregasi, bukan angka statis.

Bukti: Query agregasi dan demo dashboard ketiga role.

REPORT-01 Laporan CSV

Ketentuan minimum: - Ekspor CSV pergerakan stok (StockLedger) dan status order dalam rentang tanggal, dihasilkan dari query agregasi/rekap yang sama dengan dashboard.

Bukti: File CSV hasil ekspor dengan rentang tanggal berbeda.

2.6 API

API-01 Endpoint JSON

Minimal satu endpoint mengembalikan data dalam format JSON sebagai kontrak API, terpisah dari halaman HTML biasa.

Ketentuan minimum: - Contoh: `GET /api/products/{sku}/availability` mengembalikan stok per gudang. - Autentikasi diperiksa sama seperti halaman biasa; respons memakai `Content-Type: application/json` dan kode status http yang tepat (200/401/404) - bukan halaman HTML error.

Bukti: Demo pemanggilan endpoint dengan dan tanpa autentikasi, serta SKU tidak ditemukan.

2.7 Validation, Error Handling, UI & Database

VAL-01 Validation & Feedback

Ketentuan minimum: - Field wajib, enum status, tanggal, foreign key, dan angka (qty/harga/stok ≥ 0) divalidasi di frontend dan backend; backend adalah sumber kebenaran. - Data tidak disimpan jika validasi gagal; input yang sudah diisi dipertahankan jika relevan.

Bukti: Daftar skenario invalid input dan hasil pengujiannya.

ERR-01 Error Handling

Ketentuan minimum: - Akses tanpa login diarahkan ke login; tanpa kewenangan menampilkan 403; data/URL tidak ditemukan menampilkan 404. - Database exception dan stack trace tidak ditampilkan kepada user.

Bukti: Demo minimal dua failure path yang disengaja dan aman.

UI-01 Responsive & Usability

Ketentuan minimum: - Login, dashboard, daftar produk/order, detail, dan form dapat digunakan pada 360px dan desktop; navigasi dan tabel tidak terpotong. - Form memiliki label; focus state dan kontras dasar terlihat.

Bukti: Screenshot desktop dan mobile untuk keempat halaman utama.

DB-01 Database Relasional & Transaksi

Ketentuan minimum: - Tabel minimum sesuai §1.3; primary key, foreign key, constraint (mis. `quantity >= 0`), dan index yang relevan tersedia. - Seluruh query menggunakan PDO prepared statement; operasi multi-tabel (goods receipt, goods issue) dibungkus transaksi eksplisit. - Schema dan seed dapat membuat database dari kondisi kosong, termasuk data untuk `FIND-01`.

Bukti: ERD, schema-and-seed.sql, penjelasan satu transaksi multi-tabel dan satu index.

JOB-01 Script Terjadwal

Tugas ini dipisahkan dari siklus request web, mencerminkan proses yang di dunia nyata biasanya berjalan lewat cron.

Ketentuan minimum: - Script mandiri (mis. `php scripts/check-low-stock.php`) menghasilkan ringkasan produk di bawah reorder point. - Dapat dijalankan manual melalui `docker compose exec`; tidak wajib dijadwalkan otomatis di server penilaian.

Bukti: Demo menjalankan script dan output ringkasannya.

3. Arsitektur & Kualitas Desain

Bagian ini adalah inti penilaian level Intermediate. Yang dinilai bukan istilah pattern yang disebutkan di README, melainkan bukti bahwa peserta benar-benar memahami *mengapa* sebuah keputusan desain diambil - dan mampu mempertahankannya saat ditanya.

3.1 Layered Architecture & Dependency Inversion

ARCH-01 Pemisahan Layer & Interface pada Boundary Repository

Business logic tidak boleh bergantung langsung pada PDO, session, atau superglobal PHP.

Ketentuan minimum: - Tiga layer pragmatis: Controller (HTTP/routing) -> Service (business rule) -> Repository (akses data), dengan arah dependency dari Controller ke Repository, tidak sebaliknya. - Minimal satu interface Repository memiliki dua implementasi: implementasi MySQL asli dan implementasi in-memory/fake yang dipakai unit test. - Service menerima dependency lewat constructor (constructor injection); tidak ada `new PDO()` tersembunyi di dalam Service. - Test business logic (mis. perhitungan stok, validasi status) dapat dijalankan tanpa koneksi database sungguhan.

Bukti: Kode interface + dua implementasi, dan demo unit test Service yang berjalan memakai fake repository.

ARCH-02 Transaksi & Concurrency-Safe Stock Operation

Dua goods issue yang diproses hampir bersamaan tidak boleh menghasilkan stok negatif (oversell).

Ketentuan minimum: - Perubahan ProductStock dan penulisan StockLedger terjadi dalam satu transaksi (`beginTransaction/commit/rollback`). - Ketika dua goods issue untuk produk dan gudang yang sama diproses hampir bersamaan, hasil akhir stok harus tetap benar: tidak boleh oversell, dan tidak boleh ada update yang saling menimpa. Mekanisme untuk mencapai ini adalah bagian dari keputusan desain peserta. - Peserta dapat menjelaskan skenario konkuren yang dicegah oleh mekanisme yang dipilih, dan menunjukkan test atau skenario terkontrol yang membuktikannya - simulasi thread/paralel sungguhan tidak wajib.

Bukti: Penjelasan mekanisme yang dipilih untuk mencegah oversell, dan test/skenario yang menunjukkan permintaan kedua ditolak atau ditunda ketika stok sudah habis oleh permintaan pertama.

3.2 Class Diagram, ADR & Refactoring Log

DESIGN-01 Class Diagram - Initial & As-Built

Ketentuan minimum: - Diagram *initial* dibuat sebelum coding (`docs/planning/`), menunjukkan Controller/Service/Repository/Entity dan relasinya. - Diagram *as-built* dibuat di akhir (`docs/architecture/`), dengan penanda yang membedakan dependency mana yang mengarah ke interface dan mana yang mengarah ke kelas konkret. - Sertakan 2-3 kalimat tentang apa yang berubah antara initial dan as-built, dan alasannya. - Tool bebas (draw.io, PlantUML, Mermaid, atau sketsa yang di-scan) selama jelas dan sesuai kode.

Bukti: Kedua diagram; assessor akan menelusuri satu kelas dari diagram ke kode saat defense.

DESIGN-02 Architecture Decision Record (ADR)

Ketentuan minimum: - 2-3 ADR singkat (context / decision / consequences) untuk keputusan nyata, mis. "mengapa Repository pattern dibanding PDO langsung di controller" atau "mengapa memilih mekanisme tertentu untuk mencegah oversell pada ARCH-02."

Bukti: File `docs/architecture/adr-*.md`.

DESIGN-03 Refactoring Log, Audit SRP & Tech-Debt Register

Ketentuan minimum: - Refactoring log berisi minimal 3 entri: nama smell (Long Method, Duplicate Code, Feature Envy, Primitive Obsession, dsb.), teknik yang dipakai (Extract Method, Extract Class, Replace Conditional with Polymorphism, dsb.), dan cuplikan sebelum/sesudah. - Satu catatan audit SRP: satu kelas dari draf awal yang melanggar SRP, dan bagaimana dipecah. - Tech-debt register: keterbatasan/jalan pintas yang diambil karena waktu, dan perbaikan idealnya - dicatat jujur, bukan disembunyikan. - Minimal satu commit bertag `refactor:` yang memperbaiki kode lama, bukan fitur yang sedang dikerjakan (Boy Scout Rule).

Bukti: `docs/quality/refactor-log.md`, `docs/quality/tech-debt.md`, dan commit history.

DESIGN-04 Critique Exercise

Assessor menyediakan satu cuplikan kode yang sengaja bermasalah (mis. satu Service yang menangani validasi, penyimpanan, dan pengiriman notifikasi sekaligus).

Ketentuan minimum: - Peserta menulis kritik singkat: smell apa yang ada, prinsip SOLID apa yang dilanggar, dan bagaimana seharusnya direfaktorkan - implementasi perbaikan tidak wajib, cukup analisis tertulis.

Bukti: `docs/quality/critique.md`, dibahas singkat saat technical defense.

3.3 Testing sebagai Bagian dari Desain

TEST-01 Unit Test Terisolasi

Ketentuan minimum: - Minimal 6 test case pada minimal 3 area logika (mis. validasi tanggal PO, transisi status SO, perhitungan low-stock, ownership/authorization approve). - Tidak menyentuh session, PDO nyata, atau layanan eksternal; test getter/setter sederhana tidak dihitung.

Bukti: Hasil test di `docs/testing/`, dijalankan lewat satu perintah README.

TEST-02 Integration Test

Ketentuan minimum: - Minimal 3 integration test yang menyentuh MySQL nyata di Docker, mis. goods receipt benar-benar menambah stok end-to-end, atau goods issue kedua ditolak ketika stok sudah habis oleh goods issue pertama.

Bukti: Test terpisah dari unit test, dijalankan lewat perintah README yang sama atau terpisah dan dijelaskan.

TEST-03 Static Analysis & FIRST

Ketentuan minimum: - Laporan PHPStan (level 5+) atau PHP_CodeSniffer (PSR-12) dilampirkan; nol critical error, warning yang tersisa dijelaskan singkat. - Test mengikuti FIRST (Fast, Independent, Repeatable, Self-validating, Timely): tidak ada `sleep()`, panggilan jaringan nyata, atau ketergantungan urutan eksekusi.

Bukti: Laporan static analysis di `docs/quality/`.

4. Ketentuan Teknis

Penting Ketentuan pada bagian ini bersifat wajib. Fitur yang berjalan dengan teknologi yang dilarang tetap dianggap tidak memenuhi requirement teknis.

Area	Wajib	Diperbolehkan	Tidak diperbolehkan
Frontend	HTML semantik, CSS buatan peserta, Vanilla JS	Fetch API, library icon yang dicantumkan	React, Vue, Angular, jQuery, framework CSS, template admin siap pakai
Backend	PHP 8.2+ Native, OOP berlapis, DIP pada boundary repository	Composer untuk autoload & dev dependency	Laravel, CodeIgniter, Symfony, Slim, ORM, generator CRUD, DI container framework
Database	MySQL 8, relasi, constraint, index, PDO prepared statement, transaksi eksplisit	Migration/seed buatan peserta	NoSQL sebagai penyimpanan utama, query concatenation input user
Environment	Dockerfile, Docker Compose, .env.example	Apache/Nginx sesuai rancangan	

Area	Wajib	Diperbolehkan	Tidak diperbolehkan
			Setup yang hanya berjalan di komputer peserta
Testing	PHPUnit: unit + integration test, static analysis report	Test tambahan lain	Test trivial getter/setter, test yang hanya lulus karena di-skip
Desain	Class diagram initial & as-built, ADR, refactoring log	Tool diagram apapun yang legible	Diagram yang tidak sesuai kode aktual

4.1 Struktur Aplikasi

Nama folder boleh berbeda, tetapi tanggung jawab berikut harus dipisahkan secara masuk akal:

- `public/` - entry point & static asset.
- `app/Controller`, `app/Service`, `app/Repository`, `app/Entity` - empat tanggung jawab terpisah sesuai ARCH-01.
- `views/` - template antarmuka.
- `config/` - loader environment.
- `database/` - schema, seed.
- `tests/Unit`, `tests/Integration` - dipisah secara eksplisit.
- `docs/planning/`, `docs/architecture/`, `docs/quality/`, `docs/testing/`.

4.2 Security Minimum

- Password hashing menggunakan API password PHP; authorization selalu diperiksa di server, termasuk aturan segregation of duties pada §1.2.
- Semua query yang menerima input menggunakan prepared statement; output user di-escape sebelum ditampilkan pada HTML.
- Session dikelola aman, ID session diperbarui setelah login; secret dan credential tidak disimpan di repository.
- Operasi stok pada ARCH-02 dirancang agar aman dari kondisi balapan (race condition) yang bisa merusak data; mekanisme pencegahannya menjadi bagian dari keputusan desain peserta.

4.3 Di Luar Scope

Peserta tidak diwajibkan membuat microservices, message queue sungguhan, cloud deployment, CI/CD, Kubernetes, real-time notification, mobile application, cron scheduler otomatis, atau automated end-to-end test.

4.4 Fitur Bonus

Dinilai setelah seluruh requirement wajib stabil. Contoh: notifikasi email simulasi (mis. lewat Mailhog di Docker), audit trail perubahan master data, dashboard grafik (SVG/canvas buatan sendiri), atau integration test tambahan. Bonus tidak dapat menutup requirement wajib yang tidak berfungsi.

5. Docker & Testing

5.1 Docker Wajib

- Minimal service aplikasi/web dan service MySQL; aplikasi dibangun dari kondisi bersih memakai `docker compose up --build`.
- Konfigurasi lewat environment variable dengan contoh nilai pada `.env.example`; schema/seed dijalankan sesuai prosedur README.
- Source code tidak bergantung pada absolute path atau konfigurasi khusus komputer peserta.

Uji sebelum submission Clone/copy release final ke folder bersih, jalankan perintah pada README, lalu pastikan login, alur PO/SO, dan unit+integration test berfungsi.

5.2 Pemeriksaan Langsung

Pada demo/technical defense, assessor dapat meminta peserta menjalankan Docker, menjalankan unit dan integration test, menelusuri class diagram ke kode, mendemonstrasikan skenario concurrency pada `ARCH-02`, membahas satu ADR, atau membuat perubahan kecil pada logic yang dipilih sambil menjaga test tetap hijau (safe-refactor demo).

6. Proses Kerja, Git & Penggunaan AI

Checkpoint dipantau bersama trainer sepanjang pengerjaan; peserta menyusun sendiri urutan kerjanya mengikuti urutan pembangunan pada §2 (vertical slice dahulu, polish kemudian).

6.1 Git & Integritas Proses

- Repository individual; commit bertahap menggambarkan perubahan nyata, termasuk minimal satu commit `refactor: (DESIGN-03)`.
- Jangan memasukkan `.env`, credential aktif, token, atau data client/PII.
- Cantumkan sumber snippet, package, atau asset yang bukan buatan peserta.

6.2 Penggunaan AI

Empat kewajiban DISCLOSE penggunaan AI, REVIEW output, VERIFY dengan requirement/dokumentasi, dan TEST behavior. Peserta tetap bertanggung jawab penuh atas solusi, termasuk mampu menjelaskan setiap keputusan arsitektur tanpa bantuan AI saat defense.

- Catat tool, tujuan, ringkasan prompt yang sudah disanitasi, output yang digunakan/ditolak, dan bukti verifikasi pada `ai-usage-log.md`.
- Jangan mengirim source code proprietary, data client, atau credential ke layanan AI publik.

7. Submission Package

Artefak	Isi minimum
Source code	Kode final, composer.json/lock, .gitignore, dependency declaration
Docker	Dockerfile, compose.yaml, .env.example
README.md	Fitur, requirement, instalasi, akun demo, perintah Docker/test, known limitations
database/	schema-and-seed.sql atau migration/seed setara
tests/Unit, tests/Integration	Unit test & integration test yang memenuhi standar minimum

Artefak	Isi minimum
docs/planning/	User story, scope, ERD, class diagram initial, backlog
docs/architecture/	Class diagram as-built, ADR
docs/quality/	Refactoring log, audit SRP, tech-debt register, critique.md, laporan static analysis
docs/testing/	Test scenario, hasil test, screenshot relevan, known bugs
ai-usage-log.md	Deklarasi penggunaan AI, termasuk jika tidak menggunakan AI
Release final	Tag/release resmi; commit setelah freeze tidak dinilai kecuali diminta

7.1 Data Demo Minimum

- Satu akun Admin, minimal dua akun Sales, minimal dua akun Warehouse Staff.
- Minimal dua gudang, 30 produk dengan variasi reorder point, dan beberapa produk di bawah reorder point.
- Minimal 25 order gabungan (PO+SO) dengan variasi status, termasuk contoh yang PendingApproval dan Cancelled.

8. Demo, Penilaian & Critical Failure

8.1 Demo dan Technical Defense

Bagian	Durasi	Yang ditunjukkan
Demo produk	12-15 menit	Docker start, login 3 role, alur PO->SO->ledger, validation/failure, find/pagination, dashboard, responsive
Engineering evidence	5-7 menit	Class diagram, ADR, unit+integration test, static analysis, refactor log
Technical defense	10-12 menit	Critique exercise, penjelasan ARCH-02, tracing diagram->kode, safe-refactor demo

8.2 Critical Failure

- Aplikasi atau database tidak dapat dijalankan lewat Docker setelah prosedur setup yang wajar.

- Alur inti login-PO/SO-stok tidak berfungsi, atau fitur inti hanya tampilan tanpa proses/data nyata.
- Project menggunakan framework backend/frontend, ORM, atau DI container framework yang dilarang.
- Tidak ada unit test dan integration test yang valid, atau seluruh test gagal pada release final.
- Password plaintext, secret aktif masuk repository, query menggabungkan input user secara mentah, atau authorization hanya di frontend.
- Stok diubah langsung tanpa melalui service/ledger, sehingga StockLedger tidak konsisten dengan ProductStock.
- Goods issue/receipt tidak transaksional, sehingga oversell dapat direproduksi assessor pada saat defense.
- Class diagram tidak mencerminkan kode aktual dan tidak dapat ditelusuri saat defense.
- Peserta tidak mampu menjelaskan arsitektur/keputusan desain sendiri, atau evidence menunjukkan plagiarisme.
- Penggunaan AI atau sumber eksternal yang material sengaja disembunyikan.

Status project Nilai minimal 80 dan tidak terdapat critical failure. Kelulusan program secara keseluruhan mengikuti mekanisme training dan assessment yang berlaku.

9. Pertanyaan yang Sering Diajukan

1. **Apakah boleh menggunakan ORM seperti Eloquent standalone?** Tidak. PDO native tetap wajib; interface Repository silakan dibuat sendiri untuk mendapatkan manfaat abstraksi tanpa ORM.
2. **Apakah butuh DI container seperti PHP-DI atau Symfony DI?** Tidak wajib. Constructor injection manual sudah cukup untuk menunjukkan pemahaman Dependency Inversion.
3. **Apakah class diagram harus dibuat dengan tool UML formal berbayar?** Tidak. draw.io, PlantUML, Mermaid, atau sketsa yang discan diterima selama jelas dan sesuai kode aktual.
4. **Apakah arsitektur harus mengikuti 4 ring Clean Architecture penuh?** Tidak. Tiga layer pragmatis (Controller/Service/Repository) dengan interface pada boundary repository sudah memadai.

5. **Apakah critique exercise (DESIGN-04) wajib diimplementasikan perbaikannya?** Tidak. Analisis tertulis yang menyebutkan smell, prinsip yang dilanggar, dan arah perbaikan sudah cukup.
 6. **Berapa jumlah unit test dan integration test minimum?** Minimal 6 unit test pada ≥ 3 area, ditambah minimal 3 integration test yang menyentuh MySQL nyata di Docker.
 7. **Apakah static analysis report wajib nol error sepenuhnya?** Wajib nol critical error. Warning yang tersisa boleh ada selama dijelaskan dan dicatat, bukan diabaikan diam-diam.
 8. **Apakah race condition harus diuji dengan request paralel sungguhan?** Tidak wajib. Menjelaskan mekanisme yang dipilih untuk mencegah oversell dan membuktikannya lewat test/skenario terkontrol yang dapat diverifikasi assessor sudah cukup.
 9. **Apakah script terjadwal (JOB-01) harus benar-benar berjalan otomatis di server?** Tidak. Dapat dijalankan manual lewat perintah Docker selama tujuan pemisahannya dari request web jelas.
 10. **Apakah fitur bonus dapat menutup requirement wajib yang belum selesai?** Tidak. Bonus hanya dinilai setelah seluruh requirement wajib stabil.
 11. **Apakah harus menyediakan halaman register?** Tidak. Tidak ada public registration; seluruh akun dibuat oleh Admin.
 12. **Bagaimana jika requirement terasa ambigu?** Ajukan pertanyaan sebelum mengubah scope. Simpan jawaban trainer sebagai catatan keputusan pada docs/planning/.
-

10. Checklist Sebelum Submission

- [] Seluruh requirement wajib (§2) telah diuji pada release/tag final.
- [] Aplikasi dan database dapat dijalankan lewat Docker dari folder bersih.
- [] Unit test dan integration test dapat dijalankan dengan satu perintah dan seluruhnya lulus.
- [] Laporan static analysis terlampir dengan nol critical error.
- [] Class diagram initial dan as-built tersedia dan sesuai kode aktual.
- [] Goods issue/receipt terbukti transaksional; skenario oversell tidak dapat direproduksi.
- [] Segregation of duties (Sales tidak dapat approve order sendiri) teruji di server, bukan hanya UI.

- [] Search, filter, sort, pagination, dashboard 3 role, dan endpoint JSON dapat didemonstrasikan.
- [] Tidak ada secret, credential aktif, data client, atau PII pada repository dan history yang diserahkan.
- [] README telah diuji dari environment bersih; akun demo tiga role tersedia.
- [] Peserta dapat menjelaskan data flow, layered architecture, satu ADR, dan satu refactor tanpa bantuan AI.
- [] Release/tag final sudah dibuat dan link submission sudah benar.

Final check Jika peserta belum dapat menunjukkan atau menjelaskan satu item pada checklist, selesaikan atau catat keterbatasannya sebelum membuat release final.

Neuronworks / Learning Series - Intermediate Programmer Final Project Brief, Participant Guide Edisi 1.0. Dokumen ini menjadi acuan resmi peserta dalam merancang, membangun, menguji, dan mengumpulkan Final Project Inventory & Order Management System.